

Memory Management

1. The Memory Hierarchy

Speed ↑ Size ↓		Speed ↓ Size ↑
CPU Registers		fastest, smallest, priciest
Cache Memory		
Main Memory		RAM
Secondary Storage		slowest, largest, cheapest (disk)

A CPU **never** deals with the physical address directly — it doesn't "know" about physical RAM addresses at all. The CPU only ever generates and works with **logical (virtual) addresses**. Something else must map those to real, physical RAM locations — that "something else" is the core subject of this entire topic.

2. Address Binding

Address binding = deciding *when* a program's logical addresses get mapped to actual physical memory addresses. There are three points at which this can happen:

2.1 Compile-Time Binding

- The compiler itself decides the final **physical address** (e.g., `x → 2000`, fixed forever).
- **Limitation:** if the OS ever needs to load this program somewhere else, it can't — the program must be **recompiled**. No relocation, no multitasking flexibility.
- Think: *"This program MUST run only at this exact location."*

2.2 Load-Time Binding

- The compiler generates **relative** addresses (e.g., `x → 100`, relative to 0).
- The **loader** decides the base address at load time (e.g., `Base = 5000 → x = 5100`).
- **Limitation:** addresses become fixed once loading is done — the program can be placed anywhere *initially*, but can't move once it starts running.
- Think: *"You can choose where to start, but once started, you're stuck."*

2.3 Execution-Time (Run-Time) Binding

- The program always uses its **logical address** (e.g., 100); the **hardware** (MMU) computes $\text{Physical} = \text{Base} + \text{Logical}$ on *every single access*, not just once at load time.
- **This is what enables modern OS features:** the OS can move a process in memory, or swap it out to disk and back, all while it's running — because the mapping is recomputed every time, not baked in once.
- **Requires:** dedicated hardware — the **MMU (Memory Management Unit)**.
- Think: *"You don't know where you live — the system redirects everything dynamically, every time you ask."*

Key trade-off: compile-time binding is the fastest to resolve (no runtime cost) but the least flexible. Run-time binding is the most flexible (enables relocation, swapping, virtual memory) at the cost of needing hardware translation on every memory access.

3. The MMU (Memory Management Unit)

One-line definition: MMU = hardware that translates every logical (virtual) address the CPU generates into a physical RAM address, in real time.

Why we need it

Without an MMU, every program would need to know real physical addresses directly — programs would overwrite each other, there'd be no isolation, no security, no multitasking, and no meaningful memory protection.

The 3 problems MMU solves

1. **Relocation** — a program can be loaded anywhere in RAM; it always thinks its addresses start at 0, and the MMU handles the real mapping.
2. **Protection** — each process gets a bounded region; the MMU checks every access against that boundary.
3. **Isolation** — one process's memory accesses can never reach another process's memory, because the MMU only ever translates within the current process's allowed range.

Basic mechanism: Base + Limit registers

CPU generates logical address = 100

MMU:

Check: $100 < \text{limit}$?

Physical = Base + 100

Example: Base = 5000, Limit = 1000

- Logical 200 → $200 < 1000$ ✓ → Physical = 5200

- Logical 1500 → $1500 > 1000$ ✗ → **access denied** (a hardware-triggered trap — this is literally what causes a segmentation fault)

CPU → MMU → RAM

Every single memory access — for variables, arrays, and even instruction fetches — goes through the MMU. It's not an occasional check; it's on the critical path of every access.

Base+limit is the *simplest* mechanism. Real, modern MMUs implement this same idea via more sophisticated schemes — **paging** and **segmentation** (§6–8) — which are really just more flexible ways of answering the same question the base+limit model answers.

Interview soundbite: "The MMU is the hardware that turns every logical address the CPU generates into a physical one, checking bounds along the way — it's what makes relocation, protection, and process isolation possible without the compiler needing to know final physical addresses in advance."

4. Linking, Loading, and Swapping (context around address binding)

4.1 Linking

Combines your compiled code with external code it depends on (like library functions such as `printf`) into one complete executable, resolving "where is this function's code actually located" — a **static** copy embeds the library's code directly (larger file, no runtime dependency, faster execution); **dynamic** linking resolves it at runtime by loading a shared library (`.so` / `.dll`), giving a smaller executable, shared code across programs, but a small runtime linking cost.

Distinction to remember: *Linking* decides which function's code goes where within the final program; *address binding* decides where the overall program sits in memory.

4.2 Loading

Places the linked executable into RAM and prepares it to run (sets up the program counter, stack, registers). Loading style mirrors the address-binding choice: **absolute** (fixed physical address, no flexibility), **relocatable** (loader picks the base, adjusts addresses), or **dynamic loading** (only load specific functions/modules into memory *when actually called*, saving memory for code paths never used).

4.3 Swapping

Temporarily moves an entire process out of RAM into a reserved disk area (the **swap space / backing store**) to free RAM for other processes, and brings it back later.

- **Swap out** — RAM → Disk. **Swap in** — Disk → RAM.
- **Only works cleanly with run-time binding** — if a process's addresses were fixed at load time, it would have to swap back into the *exact same* physical location; with run-time binding, the base register is simply updated to wherever it lands next, and the process (using only logical addresses) never notices.
- **Cost:** swapping is slow, since it goes through disk. Swap heavily and frequently enough, and you get **thrashing** — the system spends more time swapping processes in/out than actually executing them (this connects directly into the Virtual Memory topic).

5. Contiguous Memory Allocation

Each process is allocated **one single continuous block** of memory.

```
RAM: | OS | P1 | P2 | P3 | Free |
```

5.1 Single Partition Allocation

Memory split into just an OS area and one user-process area — only one process can run at a time. Mostly of historical interest.

5.2 Multiple Partition Allocation

(a) Fixed (static) partitioning — memory divided into fixed-size blocks up front.

```
| OS | 100KB | 100KB | 100KB | 100KB |
```

Simple and fast to allocate, but if a 70KB process is placed into a 100KB partition, **30KB is permanently wasted inside that partition — internal fragmentation**.

(b) Variable (dynamic) partitioning — partitions are created dynamically, exactly the size each arriving process needs.

```
| OS | 120KB | 200KB | 80KB | Free |
```

Better utilization initially, but as processes come and go, memory gets chopped into scattered free chunks — **external fragmentation**: the *free space exists* but is scattered into pieces too small individually to satisfy a new request, even though their *sum* would be enough.

Example: free blocks of 50KB + 30KB + 20KB = 100KB total, but a process needing a contiguous 90KB block **cannot** be satisfied — no single block is that large, even though the total free space is.

5.3 Allocation Strategies (for variable partitioning)

Strategy	Rule	Pros	Cons
First Fit	Allocate the <i>first</i> free block large enough	Fast	Leaves many small unusable gaps over time
Best Fit	Allocate the <i>smallest</i> block that still fits	Minimizes wasted space per allocation	Slow (must scan all of memory); ironically tends to create many tiny, useless leftover fragments
Worst Fit	Allocate the <i>largest</i> available block	Leaves a large, more usable leftover chunk	Generally poor overall utilization; large blocks get consumed quickly

5.4 Compaction — the fix for external fragmentation

Shift all allocated processes together to one end of memory, merging all the scattered free space into one single contiguous block.

```
Before: | P1 | Free | P2 | Free | P3 |
After:  | P1 | P2 | P3 | Free (combined) |
```

Cost: compaction requires actually copying process memory around — this is only feasible with **run-time binding** (so addresses can be transparently updated), and it's a genuinely expensive operation to perform (you must pause/relocate potentially many processes at once).

5.5 Advantages / Disadvantages of Contiguous Allocation

- ✓ Simple to implement, fast direct access, easy bookkeeping.
- ✗ Fragmentation (internal with fixed partitions, external with variable), difficult for programs larger than any single available block, generally not how modern general-purpose OSes manage memory.

6. Non-Contiguous Allocation: Paging

A process does **not** need to sit in one continuous block — it can be split up and scattered across memory in pieces, as long as the OS keeps track of where every piece went.

6.1 Core idea

- **Page** — a fixed-size block of a *process*.
- **Frame** — a fixed-size block of *physical memory (RAM)*.

- **Page size = Frame size**, always.
- A process's pages are placed into *any* available frames — not necessarily contiguous ones.

Process P: [Page 0] [Page 1] [Page 2]
 Memory: | Frame 5 | Frame 2 | Frame 9 | ← scattered, non-contiguous

This is *exactly* how paging solves external fragmentation: since any free frame can hold any page, there's never a "free space exists but is too scattered to use" problem — a free frame is a free frame, wherever it is.

6.2 Address translation

Logical address = **(Page number, Offset)**.

```

CPU generates (Page, Offset)
      |
      v
Look up Page in Page Table
      |
      v
Get Frame number
      |
      v
Physical Address = Frame number × frame_size + Offset
  
```

Worked example: page size = 100 bytes, logical address = 250.

- Page = $250 \div 100 = 2$, Offset = **50**
- Page table says: Page 2 → Frame 7
- Physical address = $7 \times 100 + 50 = 750$

6.3 Page Table & PTBR

The **page table** stores the Page-Number → Frame-Number mapping, and lives **in RAM** (it's just another data structure the OS maintains). Since **every process has its own separate page table**, the CPU needs to know *where the current process's page table is* — that's exactly what the **Page Table Base Register (PTBR)** is for: it holds the starting address of the *currently running* process's page table.

- On a **context switch**, the OS updates PTBR to point to the new process's page table — this is *how* the CPU automatically uses the correct mapping for whichever process is now running.
- Without PTBR, the CPU would have no way to find the right page table for the currently running process.

The naive cost: every single logical memory access now takes **2 physical memory accesses** — one to read the page table entry (found via PTBR), and one to actually fetch the data. This doubles effective memory latency, which is a serious problem.

6.4 TLB (Translation Lookaside Buffer) — fixing the 2-access problem

One-line definition: the TLB is a small, very fast, hardware cache (sitting right next to the CPU) that stores recently-used Page → Frame mappings.

CPU → TLB → Main Memory

On every memory access:

- **TLB hit** — the mapping is already cached → skip the page table entirely → **1 memory access total** (just fetch the data).
- **TLB miss** — not cached → fall back to the page table (1 access), fetch the actual data (1 access), **and cache this mapping in the TLB** for next time → **2 memory accesses total** (same as without a TLB, but only on a miss).

Why TLB hit rates are typically high: programs exhibit **locality of reference** — the same handful of pages tend to be accessed repeatedly in a short window (e.g., loop bodies, the current stack frame), so recently-cached mappings are very likely to be reused.

TLB replacement: when full, evict an entry using a policy like **LRU** or **FIFO** (the same page-replacement ideas covered in the Virtual Memory topic).

6.5 Effective Access Time (EAT) — the key numerical tool

$$\text{EAT} = (\text{Hit Ratio} \times \text{time_with_hit}) + (\text{Miss Ratio} \times \text{time_with_miss})$$

Worked example (from your notes): memory access time = 100ns, TLB access time = 10ns, hit ratio = 75%.

$$\begin{aligned}\text{EAT} &= h \cdot (\text{TLB} + \text{Mem}) + (1-h) \cdot (\text{TLB} + \text{Mem} + \text{Mem}) \\ &= 0.75(10+100) + 0.25(10+100+100) \\ &= 0.75(110) + 0.25(210) \\ &= 82.5 + 52.5 \\ &= 135 \text{ ns}\end{aligned}$$

(This is the same style of numerical your notes use — always double-check whether the question's timing model counts the TLB check as adding to, or overlapping with, the memory access, since that changes the formula slightly.)

6.6 Fragmentation in Paging

- **External fragmentation — eliminated.** Any free frame fits any page; there's no "scattered but unusable" free space problem.
- **Internal fragmentation — still exists**, in the *last* page of a process. If page size = 100 bytes and a process needs 270 bytes, it needs **3 pages** (300 bytes allocated) — **30 bytes wasted** in that final, partially-used page.

6.7 Advantages / Disadvantages of Paging

-  No external fragmentation, efficient use of memory, straightforward allocation (any free frame works), and it's the foundation virtual memory (Virtual Memory topic) is built on.
-  Internal fragmentation (bounded by page size, typically small), page table memory overhead (especially for large address spaces — mitigated by multi-level/hierarchical page tables in practice), and address translation cost (mitigated by the TLB).

7. Memory Protection

One-line definition: memory protection prevents a process from accessing memory it isn't allowed to touch — enforced through extra bits in each page table entry.

7.1 Valid/Invalid bit

Each page table entry carries a bit:

- **Valid (1)** — this page genuinely belongs to the process (and, in virtual memory systems, is currently loaded).
- **Invalid (0)** — this page does **not** belong to the process, **or** it's a legitimate page that simply isn't loaded in RAM right now (relevant later, in demand paging).

If the CPU generates an address whose page has `Valid = 0`, the MMU raises a **trap (exception)** — this is exactly what a **segmentation fault** is under the hood.

Important nuance: "invalid" doesn't automatically mean "buggy program." It can also legitimately mean "this page exists but isn't currently in memory," which the OS handles as a page fault rather than an error, if the page genuinely belongs to the process (see the Virtual Memory topic).

7.2 Read / Write / Execute bits

Beyond valid/invalid, real systems attach **permission bits** to each page/segment:

Bit	Meaning
R	Read allowed
W	Write allowed
X	Execute allowed

Typical usage: code segments get **R+X** (you can run it, read it, but not modify it); data segments get **R+W** (you can read/write it, but code shouldn't be executed *from* it — this is exactly the protection that prevents classic buffer-overflow exploits from executing injected code sitting in a writable data/stack region).

Interview soundbite: "Memory protection is enforced per page via a valid/invalid bit (does this page belong to me, and is it present?) plus read/write/execute permission bits, all checked by the MMU on every access — a violation triggers a hardware trap that the OS turns into a segmentation fault or a page-fault handler, depending on the cause."

8. Shared Paging (and Copy-on-Write)

One-line definition: shared paging lets multiple processes' page tables point to the **same physical frame**, so the data/code only exists once in RAM even though several processes are "using" it.

```
Process A's page table: Page 1 → Frame 10
Process B's page table: Page 3 → Frame 10 ← same frame, different logical pages
```

Why: many processes load the *same* code — shared libraries, common runtime code (`printf` , etc.). Without sharing, each process would carry its own redundant copy, wasting RAM.

Condition for safe sharing: the shared page must be either **read-only**, or carefully synchronized if writable — otherwise one process's write would unexpectedly corrupt what another process sees.

Copy-on-Write (COW)

An elegant extension: initially let multiple processes **share** a page (e.g., right after `fork()` , parent and child initially share all pages). If either process tries to **write** to a shared page, *only then* does the OS copy that specific page into a new, private frame for the writer — the other process keeps using the original, unmodified frame.

```
Initially:      A → Frame 5, B → Frame 5      (shared)
After A writes: A → Frame 7 (private copy), B → Frame 5 (unchanged)
```

Why this matters: it avoids the cost of copying data that might never actually be modified — you only pay the copy cost exactly when (and if) it's actually needed. This is precisely why `fork()` is cheap in practice despite conceptually duplicating an entire address space.

9. Segmentation

One-line definition: segmentation divides a program into **logical**, meaningful units (segments) — not equal-size chunks — and stores each separately.

```
Segment 0 → Code
Segment 1 → Data
Segment 2 → Stack
Segment 3 → Heap
```

Key difference from paging: paging divides memory mechanically by *size*; segmentation divides a program by *meaning* — each segment is a natural unit like "the code" or "the stack," and different segments can be (and usually are) different sizes.

Address translation

Logical address = (**Segment number, Offset**).

Segment Table:

Segment	Base	Limit
0	1000	400
1	2000	300

Access (Segment 1, Offset 200):

$200 < 300 \checkmark \rightarrow \text{Physical} = 2000 + 200 = 2200$

Access (Segment 1, Offset 400):

$400 > 300 \times \rightarrow \text{Trap (segmentation fault)}$

Why segmentation is useful

1. **Matches program structure** — code, data, and stack are naturally separate concerns.
2. **Better protection, per segment** — e.g., mark the code segment read+execute, the data segment read+write, independently.
3. **Easy, meaningful sharing** — you can share just the *code* segment between processes running the same program, without needing to reason about which arbitrary fixed-size pages happen to contain code vs. data.

Fragmentation in Segmentation

- **External fragmentation** — **exists** (segments are variable-sized, so this has the exact same scattered-free-space problem as variable partitioning in §5.2).
- **Internal fragmentation** — **minimal/none** (a segment is sized to exactly fit what it holds).

Paging vs. Segmentation

Feature	Paging	Segmentation
Division basis	Fixed size	Logical / variable size
Fragmentation	Internal	External
Visibility	Invisible to the programmer (pure physical-memory mechanism)	Visible to the programmer (reflects logical program structure)

Note the complementary weaknesses: paging fixes external fragmentation but has internal fragmentation and no logical meaning; segmentation is logically meaningful and has no internal fragmentation but suffers external fragmentation. This complementary pairing is exactly why the next scheme exists.

10. Paged Segmentation (Hybrid Approach)

One-line definition: each segment is further divided into fixed-size pages — combining segmentation's logical structure with paging's fragmentation-free allocation.

Program → Segments → Pages → Frames (in RAM)

Logical address now has **three** parts: **(Segment number, Page number, Offset)**.

Data structures

- **Segment table** — per process; each entry gives the **base address of that segment's own page table**, plus a limit (number of pages in the segment).
- **Page table** — one per segment; maps that segment's pages to physical frames, exactly like ordinary paging.

Address translation

```
CPU generates (Segment, Page, Offset)
|
Segment Table lookup → validate segment, get base of that segment's page table
|
Page Table lookup (within that segment) → get Frame number
|
Physical Address = Frame × page_size + Offset
```

Worked example: Segment 0's page table lives at address 5000. Page table: Page 0→Frame 10, Page 1→Frame 5, Page 2→Frame 20.

Access (Segment 0, Page 1, Offset 50) :

1. Segment 0 → page table at 5000.
2. Page 1 → Frame 5.
3. Physical = $5 \times \text{page_size} + 50$.

Advantages / Disadvantages

✓ No external fragmentation	Paging (within each segment) solves it
✓ Logical structure preserved	Segments still map to meaningful program units
✓ Segment-level protection & sharing	Each segment can have its own permissions
✗ Complex	Two-level lookup (segment table → page table)
✗ Slower without a TLB	Multiple memory accesses per translation

TLB relevance here is even higher: the TLB can cache (Segment, Page) → Frame directly, skipping *both* lookups on a hit — without it, every miss costs multiple sequential memory accesses (segment table, then page table, then the actual data).

Comparison Summary

Feature	Paging	Segmentation	Paged Segmentation
Division	Fixed	Logical	Logical + Fixed
Fragmentation	Internal	External	Internal only (minimal)
Complexity	Medium	Medium	High
Flexibility	Medium	High	Very High

Interview Questions With Answers

Q1. What are the three types of address binding, and what's the core trade-off between them?

Answer: Compile-time (physical address fixed by the compiler — no flexibility, fastest), load-time (relative addresses fixed once at load time by the loader — some placement flexibility, but frozen after that), and execution-time/run-time (logical addresses translated by hardware on every single access — full flexibility, including relocation while running, at the cost of needing an MMU and paying translation overhead on every access).

Q2. Why does swapping require run-time (execution-time) address binding to work well?

Answer: Because a swapped-out process, when brought back into RAM, will very likely land at a *different* physical location than before. With run-time binding, the process only ever uses logical addresses, and the MMU/base-register is simply updated to reflect wherever it landed — the process itself never needs to know or care. With compile-time or load-time binding, the process's addresses are already fixed, so it would have to be swapped back into the *exact same* physical location every time, which is far too restrictive in practice.

Q3. What three problems does the MMU solve, and how?

Answer: Relocation (a program can be loaded anywhere; it always uses addresses starting from 0 logically, and the MMU adds the base offset), protection (the MMU checks every logical address against the process's limit before translating, rejecting out-of-bounds accesses), and isolation (a process's translations are confined to its own base/limit or page table, so it structurally cannot generate a physical address inside another process's memory).

Q4. What's the difference between internal and external fragmentation? Which allocation schemes suffer from which?

Answer: Internal fragmentation is wasted space *inside* an allocated block that's larger than what's actually needed (e.g., a 100KB fixed partition holding a 70KB process wastes 30KB inside it). External fragmentation is wasted space *between* allocations — free memory exists, but it's scattered into pieces too small individually to satisfy a new request. Fixed partitioning and paging suffer internal fragmentation; variable partitioning and segmentation suffer external fragmentation.

Q5. Compare First Fit, Best Fit, and Worst Fit allocation strategies.

Answer: First Fit allocates the first free block large enough — fast, but leaves many small gaps over time. Best Fit allocates the smallest block that still fits — minimizes wasted space per individual allocation, but is slower (must scan all of memory) and ironically tends to leave many tiny, unusable leftover fragments. Worst Fit allocates the largest available block, leaving a bigger, more usable remainder each time, but tends to consume large blocks quickly and generally gives poor overall utilization.

Q6. What is compaction, and why is it expensive?

Answer: Compaction shifts all currently allocated processes together in memory to merge all scattered free space into one contiguous block, solving external fragmentation directly. It's expensive because it requires actually copying potentially many processes' memory contents to new locations, which takes time proportional to how much data must move, and it only works cleanly with run-time address binding so that each moved process's addresses can be transparently updated without the process noticing.

Q7. Walk through address translation in paging, from logical address to physical address.

Answer: The CPU generates a logical address as (page number, offset). The page number is used to look up an entry in the process's page table (located via the PTBR) to find the corresponding frame number. The physical address is then computed as $\text{frame number} \times \text{frame size} + \text{offset}$. If a TLB is present, the page-to-frame mapping is checked there first — a hit skips the page table lookup entirely.

Q8. Why do we need a PTBR, and what happens to it during a context switch?

Answer: Every process has its own separate page table, stored somewhere in RAM at a location that differs per process. The PTBR tells the CPU/MMU where the *currently running* process's page table starts, so it can perform translations correctly. During a context switch, the OS updates the PTBR to point to the newly scheduled process's page table — this is exactly the mechanism that makes the CPU automatically use the correct address mapping after switching processes.

Q9. Explain how a TLB improves paging performance, including both the hit and miss cases.

Answer: Without a TLB, every logical memory access costs two physical memory accesses — one to read the page table entry, one to read the actual data. The TLB is a small, fast hardware cache storing recently-used page→frame mappings. On a **TLB hit**, the mapping is already cached, so only one memory access (the actual data) is needed. On a **TLB miss**, the page table must still be consulted (1 access) and the data fetched (1 access) — 2 accesses total, same as without a TLB — but the mapping is then cached in the TLB for future accesses. Because programs exhibit locality of reference, hit rates tend to be high in practice, so the *average* effective access time ends up much closer to 1 access than 2.

Q10. How is memory protection actually enforced at the hardware level in a paged system?

Answer: Via bits stored in each page table entry: a valid/invalid bit indicates whether that page genuinely belongs to the process (and is currently loaded), and read/write/execute bits control what operations are permitted on that page. On every access, the MMU checks these bits before allowing the access to proceed; a violation (invalid page, or a disallowed operation like writing to a read-only page) triggers a hardware trap, which the OS turns into a segmentation fault (illegitimate access) or a page fault handler (legitimate page just not currently loaded).

Q11. What is Copy-on-Write, and why does it make `fork()` efficient?

Answer: Copy-on-Write initially lets a parent and child process (after `fork()`) share all the same physical frames rather than duplicating the entire address space immediately. Only when either process actually tries to **write** to a shared page does the OS step in and create a private copy of just that one page for the writer, leaving the other process's mapping untouched. This makes `fork()` efficient because the (often large) cost of actually copying memory is deferred and only paid for the specific pages that end up being modified — many pages (like the code segment) may never be written to at all and so are never actually duplicated.

Q12. Compare paging and segmentation directly. Why does neither one alone offer the "best of both"?

Answer: Paging divides memory into fixed-size, physically-motivated chunks — this eliminates external fragmentation (any free frame fits any page) but introduces internal fragmentation (wasted space in the last, partially-filled page) and has no relationship to the program's logical structure. Segmentation divides a program into logical, meaningful, variable-sized units (code, data, stack) — this gives essentially no internal fragmentation and enables meaningful per-segment protection/sharing, but reintroduces external fragmentation because segments are variable-sized, just like variable partitioning. Neither offers both benefits simultaneously, which is exactly the gap paged segmentation is designed to close.

Q13. How does paged segmentation combine the two, and what's the cost?

Answer: Each segment (a logical unit) is itself further divided into fixed-size pages. The logical address becomes (segment number, page number, offset): the segment number looks up that segment's own page table (via the segment table), and the page number within that page table gives the frame, exactly like ordinary paging. This eliminates external fragmentation (paging handles allocation within each segment) while preserving logical structure and segment-level protection/sharing. The cost is complexity — every translation now requires two sequential lookups (segment table, then that segment's page table) instead of one, which is noticeably slower without a TLB caching the combined (segment, page) → frame mapping directly.

Q14. Scenario: A process's page table entry for the page it's trying to access has its valid bit set to 0. Does this always mean the program has a bug?

Answer: No. A valid bit of 0 has two possible meanings: (1) the page genuinely doesn't belong to the process at all — this *is* a bug/illegitimate access, and the OS will terminate the process with a segmentation fault, or (2) the page legitimately belongs to the process, but simply isn't currently loaded into a physical frame — common in demand-paged virtual memory systems, where the OS handles this as a page fault by fetching the page from disk, rather than treating it as an error. The distinction is made by additional bookkeeping the OS keeps beyond just the single bit (e.g., whether that page is a legitimate part of the process's address space at all).